

Obsah

1.....	2
2	Hlavičky (headers) v PHP.....2
3	Přidání dat z jiné tabulky.....5
3.1.....	Příklad Select 5
4	Výpis.....6
5	Příklad opravy dat pomocí roletky 9
6	Zobrazení dat z více tabulek. 12
6.1.....	Prakticky v php 13
6.2.....	Zobrazení pomocí join 13
6.3.....	Zobraz příjmení autorů a název knihy 13
6.4.....	Vnitřní spojení 14
6.5.....	Vnější spojení 15
6.6.....	Vícenásobná spojení 16
6.7.....	Význam spojení 16
6.8.....	CROSS JOIN - kartézský součin 16
6.9.....	Příklad 16
7	Formát PDF.....19
7.1.....	Možnosti vytváření PDF v PHP 21
8.....	21
8.1.....	FPDF 21
8.2.....	FPDF –Vytvoření českých fontů 1/2 22
8.3.....	FPDF –Ahoj světe! 22
8.4.....	FPDF –Základní funkce 23
8.5.....	FPDF –Buňky 24
8.6.....	FPDF –Buňky –Kód 1/2 25
8.7.....	FPDF –Záhlaví/zápatí 27
8.8.....	Praktické příklady více tabulek a PDF 29
9	Zabezpečení stránek.....32
9.1.....	Zabezpečení pomocí hlavičky 32
9.2.....	Session 32

	9.3.....	Základní příkazy
	33	
	9.4.....	33
	9.5.....	Praktický příklad session
	33	
	header("Location: private/index.php"); - redirect to index.php.....	33
	Finish	35
	9.6.....	Šifrování hesel pomocí md5
	35	
10	Čeština a database.....	35
	10.1.....	Poznámka
	36	
11.....		36
12	urlencode.....	36

1

2 Hlavičky (headers) v PHP

Patří pod protokol HTTP.

Hlavička je v doplňkem každé odpovědi/požadavku.

Čtení hlaviček

Hlavičky jsou uloženy v proměnných, se kterými můžeme dále pracovat.

Například hlavička Referer bude přístupná v proměnné **\$HTTP_REFERER**, hlavička User-Agent bude přístupná v proměnné **\$HTTP_USER_AGENT**.

Jak je patrné, před název hlavičky připojujeme **\$HTTP_** a nahrazujeme znaky - podtržítkem _.

Dalším způsobem je zjištění všech hlaviček funkcí **GetAllHeaders()**, která vrací asociativní pole se všemi hlavičkami. Toto jde pouze pod Apachem.

```
$headers = GetAllHeaders();
while (list($hlavicka, $obsah) = each($headers)):
echo "$hlavicka - $obsah<br>";
endwhile;
```

Generování hlaviček

Generovat hlavičky musíme ještě před zasláním jakéhokoliv textového výstupu do prohlížeče. Používáme při tom funkci **Header()**. Jako parametry funkce uvedeme zápis hlavičky, který můžete vidět níže.

Tedy např. Header("Location: http://www.interval.cz/index.asp").

Hlavičky označené * (hvězdičkou) jsou dostupné jen v HTTP/1.1 protokolu.

Zápis můžete vložit přímo jako parametr do funkce Header().

Název	Popis	Zápis
Cache-Control *	Pomocí této hlavičky můžeme přikázat cache-paměti a proxy-serverům, aby konkrétní dokument neukládali do vyrovnávací paměti.	Cache-Control: no-cache
Connection *	Tato hlavička dokáže uzavřít spojení, když se pošle jako obsah hlavičky close.	Connection: close
Pragma	Pomocí této hlavičky můžeme, jako u hlavičky Cache-Control, přikázat proxy serverům a cache pamětím, aby dokument neukládali do cache-paměti	Pragma: no-cache
Accept *	Tato hlavička obsahuje informace o typech dat podporovaných klientem. Při každém typu můžeme ještě určit akceptovatelnost parametrem q , od nuly do jedna. Např. q=0.5 je lepší než q=0.2.	Accept: text/html, image/gif; q=0.4, image/png; q=0.2
Accept-Charset *	Pomocí této hlavičky můžeme určit, které znakové sady podporuje náš prohlížeč, které chceme vidět. Táto hlavička se odesílá také s parametrem q, který obsahuje akceptovatelnost, nebo oblíbenost konkrétní znakové sady.	Accept-Charset: windows-1250; q=0.7, us-ascii; q=0.3
Accept-Language *	Touto hlavičkou můžeme "sdělit" serveru, jaký jazyk preferujeme.	Accept-Language: cs, en; q=0.4
From	Hlavička obsahující elektronickou adresu	\$HTTP_FROM;

	uživatele, není už dnes podporovaná většinou prohlížečů, kvůli ochraně údajů.	
Host	Tato hlavička obsahuje doménovou adresu uživatele.	\$HTTP_HOST;
If-Modified-Since	Tuto hlavičku odesíláme, když chceme dokument dostat, jen když byl od daného datumu změněný.	If-Modified-Since: Fri, 22 Jun 2001 17:04:05 GMT
Referer	Tato hlavička obsahuje url stránky, odkud uživatel přišel, teda referera. Velmi užitečná a oblíbená hlavička.	\$HTTP_REFERER;
User-Agent	User-Agent posílá identifikaci prohlížeče, tedy svoje jméno, číslo verze, a platformu, na které je spuštěný.	User-Agent: Mozilla/4.0 (compatible; MSIE 5.01, Windows NT)
Location	V této hlavičce server odesílá url dokumentu, který byl přesunutý. Taktéž s ní můžeme uživatele automaticky přesměrovat. URL v hlavičce Location musí být absolutní, tedy musí začínat na http://	Location: http://www.interval.cz/
Server	V této hlavičce posílá server svoji identifikaci, tedy jméno a číslo verze.	Server: Apache/1.4b2
Content-Type	Hlavička obsahuje typ dat. Pro každý typ dat máme přesný zápis. Teda např. text/html - pro HTML, pro obyčejný text text/plain, pro obrázek Gif image/gif, atd.	Content-Type: text/html
Expires	V této hlavičce určujeme, po jakou dobu bude odpověď platná. Když uvedeme jako parametry aktuální čas, dokument se nebude ukládat do cache paměti.	Expires: Fri, 22 Jun 2001 17:18:17 GMT
Last-Modified	Tato hlavička obsahuje	\$HTTP_LAST_MODIFIED

	datum poslední modifikace dokumentu poslaného jako odpověď.	
--	---	--

Tedy:

Pokud chceme, aby se náš dokument neukládal do cache paměti, na začátku skriptu, před textovým výstupem, napíšeme následující: **Header("Pragma: no-cache")**, resp: **Header("Cache-control: no-cache")**, anebo uložíme do hlavičky Expires aktuální čas **Header("Expires: ".GMDate("D, d m Y H:i:s")." GMT")**.

3 Přidání dat z jiné tabulky

Příklad Select

```
<HTML>
<HEAD>
<TITLE></TITLE>
</HEAD>
<BODY>
<FORM ACTION="str104.php" METHOD="POST" >
```

Vyberte si formát, ve kterém chcete zvolená čísla zaslat

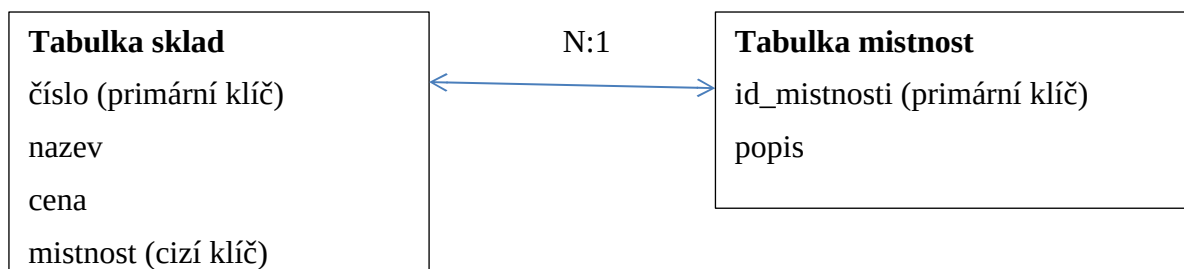
```
<SELECT NAME="Format" SIZE=1 >
<OPTION VALUE="HTML">HTML
<OPTION VALUE="PDF" SELECTED>PDF
<OPTION VALUE="PS">PS
<OPTION VALUE="ASCII">obyčejný text
</SELECT>
```

```
</BODY>
```

```
</HTML>
```

Příklad vložení dat do roletky

<http://195.113.207.163/~smrcka/tis/dbmysql2tabpdf/>




```

<TABLE>
<TR><TD>Osobni cislo:<TD><INPUT NAME=cislo VALUE="<?php echo$OC?>"
SIZE=4>
<TR><TD>Nazev:<TD><INPUT NAME=nazev>
<TR><TD>Cena:<TD><INPUT NAME=cena>
<TR><TD>Mistnost:<TD>

<select size="1" name="mistnost">

<?php
    @$vysledek = mysql_query("select * from mistnost");
    // @ potlačí zobrazení chyb
    while ($zaznam = mysql_fetch_Array($vysledek)): ?>
        <option value="<?php echo $zaznam["id_mistnosti"]?>"><?php echo
            $zaznam["id_mistnosti"]. " ". $zaznam["popis"] ?></option>
    <?php endwhile;?>
</select>

</TABLE>
<P><INPUT TYPE=SUBMIT VALUE="Pridej">
</FORM>
<FORM ACTION="index.php">
<INPUT TYPE=SUBMIT VALUE="Zpet">
</FORM>

```

4 Výpis

```

<?php

    if (!IsSet($nazev)) $nazev="";
    if (!IsSet($orderby)) $orderby="";

    require("connect.php");
?>

<HTML>
<HEAD>
<TITLE>Sklad</TITLE>
</HEAD>
<BODY>
<H1>Sklad</H1>
Hledani podle začátku názvu<BR>

<FORM ACTION=index.php method=get>

```

```
<INPUT TYPE=HIDDEN NAME=orderby VALUE="<?php echo $_GET[orderby]?>">
<INPUT TYPE=SUBMIT VALUE="hledj">
</FORM>
```

<?php

```
function TlacitkoProRazeni($polozka,$popis)
```

```
global $Nazev;
```

"<A HREF='index.php?orderby=\$polozka&Nazev='.

"<A HREF='index.php?orderby=\$polozka+DESC&Nazev='.

$$\}$$

```
// spojeni s databazi
```

```
if ($_GET[Nazev]!="")
```

```
$Podminka="WHERE nazev LIKE '".AddSlashes($_GET[Nazev])."%";
```

else

```
$Podminka = " ";
```

```
if($_GET[orderby]!="")
```

```
$OrderBy = "ORDER BY $_GET[orderby]";
```

else


```
$Orderby = "ORDER BY nazev";
```

```
$sql = "select * from sklad ".$Podminka.$Orderby;
```

```
$vysledek = mysqli_query($spojeni,$sql);
```

```
echo "<TABLE BORDER=0 CELLSPACING=0 CELLPADDING=4>\n";
```

```
echo "<TR BGCOLOR=TEAL VALIGN=TOP>\n";
```

```
echo "<TH>".TlacitkoProRazeni("cislo","Číslo")."</TH>\n";
```

```
echo "<TH>".TlacitkoProRazeni("nazev","Název")."</TH>\n";
```

```
echo "<TH>".TlacitkoProRazeni("cena","Cena")."</TH>\n";
```

```
echo "<TH>".TlacitkoProRazeni("mistnost","Místnost")."</TH>\n";
```

```
echo "<TH><TH>\n";
```

```
if (mysqli_num_rows($vysledek) > 0)
```

```
{
```

```
while ($zaznam = mysqli_fetch_assoc($vysledek)):
```

```
if (($i%2)==1) // sude aliche radky maji jinou platnost
```

```
    echo "<TR VALIGN=TOP BGCOLOR=SILVER>";
```

```
else
```

```
    echo "<TR VALIGN=TOP>";
```

```
$oc=$zaznam["cislo"];
```

```
echo "<TD ALIGN=CENTER>".$zaznam["cislo"]. "</TD>";
```

```
echo "<TD ALIGN=CENTER>".$zaznam["nazev"]. "</TD>";
```

```
echo "<TD ALIGN=CENTER>".$zaznam["cena"]. "</TD>";
```

```
echo "<TD ALIGN=CENTER>".$zaznam["mistnost"]. "</TD>";
```

```
echo "<TD ALIGN=CENTER>". "<A HREF='smazat.php?oc=$oc'>Smazat</A></TD>";
```

```
echo "<TD ALIGN=CENTER>". "<A HREF='upravit.php?oc=$oc'>Upravit</A></TD>";
```

```
echo "<TR VALIGN=TOP>";
```

```
$i=$i+1;
endwhile;
} else {
    echo "0 nalezených záznamů";
}
```

```
echo"</TABLE>";
```

```
mysqli_close($spojeni);
```

```
?>
```

```
<P><A HREF="vypis.php">Výpis ze 2 tabulek</A> <br>
```

```
<P><A HREF="vypispdf.php">Výpis do pdf</A> <br>
```

```
</BODY>
```

```
</HTML>
```

5 Příklad opravy dat pomocí roletky

```
<?php
```

```
    require("connect.php");
```

```
?>
```

```
<HTML>
```

```
<HEAD><TITLE>Uprava udaju</TITLE>
```

```
<meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />
```

```
</HEAD>
```

```
<BODY>
```

```

<?php
$sql = "select * from sklad where cislo=$_GET[oc]";
$vysledek = mysqli_query($spojeni, $sql);

//@$vysledek = mysqli_query("select * from sklad2 where cislo=$_GET[oc]");

//$pocet=mysqli_num_rows($vysledek);
$radeks = mysqli_fetch_assoc($vysledek)

//$OC = $radeks["cislo"];
//$Nazev= $radeks["nazev"];
//$cena = $radeks["cena"];
//$Mistnost = $r[3];

//endwhile;

?>
<H1>Úprava údajů položky</H1>

<!-- vypsaní položek záznamu do formuláře pro úpravy -->
<FORM ACTION="update.php" METHOD=GET>
<TABLE>
<?php echo "číslo je ". $radeks["cislo"];
?>

<TR><TD>Nazev: <TD><INPUT NAME=nazev VALUE="<?php echo $radeks["nazev"] ?
>"SIZE=11>

<TR><TD>Cena: <TD><INPUT NAME=cena VALUE="<?php echo $cena =
$radeks["cena"] ?>"SIZE=20>

<TR><TD>Místnost<td>

<select name="mistnost">

```

```

<?php
    //$query = "SELECT id_mistnost, nazev FROM mistnost";
    //$data = mysql_query($query);

    $sqlm = "SELECT * FROM mistnost";
    $vysledek = mysqli_query($spojeni, $sqlm);

    while($radek = mysqli_fetch_assoc($vysledek))
    {
        // echo "radek je ".$radek["nazev"];

        ?>
        <option value="<?php echo $radek["id_mistnosti"];?>"
        <?php if($radek["id_mistnosti"]==$radek["mistnost"]) echo
"selected"; ?>
        >

        <?php echo $radek["id_mistnosti"]." ".
$radek["popis"];?>
        </option>
    <?php
    }

    ?>
</select>

</TABLE>
<INPUT TYPE=hidden name=cislo VALUE="<?php echo $radek["cislo"];?>">

<INPUT TYPE=SUBMIT VALUE="odešli">
</FORM>

<FORM ACTION="index.php">
<INPUT TYPE=SUBMIT VALUE="Zpět">

```

```

</FORM>

<?php
mysqli_close($spojeni);
?>

</BODY>
</HTML>

```

6 Zobrazení dat z více tabulek

..Výběr dat z více tabulek

Autori

id_autora (PK)	jmeno	prijmeni
ca	Karel	Čapek
Dr.....	Jan	Drda
aj	Alois	Jirásek

Knihy

Nazev (PK)	id_autora (cizí klíč)	zanr
RUR	ca	román
Babička	bn	román
Psohlavci	aj	román

Zobraz příjmení autorů a název knihy

Zobrazení pomocí rovnosti klíčů

```

select autori.prijmeni,knihy.nazev from autori,knihy where
knihy.id_autora= autori.id_autora

```

6.1 Prakticky v php

```
<?php
```

```
require("connect.php");
```

```
$sql = "select sklad.cislo as scislo, sklad.nazev as nazevSkladu , sklad.cena as scena ,  
(mistnost.popis) as popisMistnosti FROM sklad, mistnost where  
sklad.mistnost=mistnost.id_mistnosti";
```

```
//$sql = "select sklad.cislo, (sklad.nazev) as nazevSkladu , (sklad.cena) as scena ,  
//mistnost.popis) as popisMistnosti FROM sklad LEFT join mistnost on  
//sklad.mistnost=mistnost.id_mistnosti";
```

```
//$sql = "select (sklad.nazev) as snazev FROM sklad";  
$vysledek = mysqli_query($spojeni, $sql);
```

```
//$vysledek = mysqli_query("select sklad2.cislo, (sklad2.nazev) as nazevSkladu, sklad2.cena,  
(mistnosti2.nazev) as nazevMistnosti FROM sklad2, mistnosti2 where  
sklad2.mistnost=mistnosti2.id_mistnost");  
//$vysledek = mysqli_query("select * FROM sklad2,mistnosti2 where sklad2.mistnost =  
mistnosti2.id_mistnost");
```

```
echo "<B>Výpis hodnot ze 2 tabulek</B><BR>";
```

```
while ($zaznam = mysqli_fetch_assoc($vysledek)):
```

```
    echo "Číslo: ".$zaznam["scislo"].", ";
```

```
    echo "Název: ".$zaznam["nazevSkladu"].", ";
```

```
    echo "Cena: ".$zaznam["scena"].", ";
```

```
    echo "Místnost: ".$zaznam["popisMistnosti"]." <BR>";
```

```
endwhile;
```

```
mysqli_close($spojeni);
```

```
?>
```

```
<BR><BR>
```

```
<A HREF="index.php">Prohlížení všech položek skladu</A>Zobrazení pomocí join
```

```
select autori.prijmeni, knihy.nazev from autori inner join knihy on knihy.id_autora =  
autori.id_autora
```

6.2 Zobraz příjmení autorů a název knihy

```
select autori.prijmeni,knihy.nazev from autori,knihy where knihy.id_autora= autori.id_autora  
and autori.prijmeni="Čapek"
```

```
select autori.prijmeni, knihy.nazev from autori inner join knihy on knihy.id_autora =  
autori.id_autora where autori.prijmeni="Čapek"
```

```
);
```

6.3 Vnitřní spojení

```
select autori.prijmeni, knihy.nazev from  
autori, knihy where knihy.id_autora =  
autori.id_autora;
```

Toto spojení je nazýváno vnitřní, protože jsou zobrazena pouze data, která si v obou tabulkách přesně odpovídají. Jinými slovy, knihy bez uvedeného žánru se ve výsledné sadě záznamů nikdy neobjeví, protože podmínka where to zkrátka neumožňuje. Vnitřní spojení se dá jinak (a používá se to mnohem častěji) přepsat v MySQL následujícím způsobem:

```
select autori.prijmeni, knihy.nazev from autori inner join knihy on knihy.id_autora =  
autori.id_autora;
```

```
select autori.prijmeni as prijmeni1, knihy.nazev from autori inner join knihy on  
knihy.id_autora = autori.id_autora;
```

```
$pole=mysqli_fetch_Array($vysledek)
```

echo \$pole[„prijmeni1“]

Pozn.: Klíčové slovo inner se smí vynechat, takže se často setkáte se zápisem from [název tabulky] join [název tabulky].

Jestliže by dotaz vyžadoval jak spojení tabulek, tak i nějakou omezující podmínku (případně řazení), klauzule FROM ... INNER bude v dotazu jako první, potom následuje podmínka WHERE a nakonec řazení. To bychom si měli jistě ukázat na nějakém příkladu:

```
select autori.prijmeni, knihy.nazev from
autori join knihy on knihy.id_autora =
autori.id_autora where autori.prijmeni="Čapek"
```

Vnitřní spojení se používají všude tam, kde potřebujeme mít data omezená na záznamy totožné v obou tabulkách. Co však, jestliže by v našem příkladu nějaká kniha neměla zadaného autora, a my bychom ji přesto chtěli ve výsledné sadě záznamů mít? Je to například na záznamu Hrdina Nik.

```
insert into knihy (nazev) values ('kniha bez
žánru');
```

Pro zopakování - tato kniha má zadán pouze název, takže MySQL do dalších polí v tomto záznamu doplní výchozí hodnoty. V tabulce knih existují ještě další dvě pole - "id" a "druh". Do pole "id" doplní databázový stroj nejbližší vyšší číslo (nejspíš pětku, protože zatím jsme měli 4 knihy), a do pole "druh" dá hodnotu NULL, protože jsme toto pole neurčili a hodnoty NULL v tomto sloupci smí být.

V takovém případě bychom museli při zobrazování sáhnout po trochu jiné konstrukci.

6.4 Vnější spojení

Nejprve si můžete zkusit použít trik z minulého dílu; nakonec ale zjistíte, že na úlohy tohoto typu to zkrátka nestačí. Například následující příkaz nebude fungovat tak, jak bychom potřebovali:

```
select knihy.id, knihy.nazev, druhy.nazev as
druh,
max_doba_vypujcky from knihy, druhy where
knihy.druh = druhy.id or knihy.druh is null;
```

Abychom zahrnuli všechny knihy s jejich žánry a zároveň knihy, které žádný žánr nemají, k tomu máme v MySQL klauzuli LEFT JOIN (případně RIGHT JOIN). Napsat by se to dalo asi takto:


```
select knihy.id, knihy.nazev, druhy.nazev as
druh,
max_doba_vypujcky from knihy left join druhy on
knihy.druh = druhy.id; 7
```

Všimněte si, že od původního příkazu s JOIN se tento liší pouze uvedením slova LEFT. Toto kouzelné slovíčko má za následek, že se zahrnou všechna data z tabulky knih, a z tabulky druhů (žánrů) se zahrnou pouze související data. Ve sloupcích, v nichž "párová" data v související tabulce nejsou budou vloženy hodnoty NULL.

Vnější spojení mohou být "levá" nebo "pravá". Funkčně se ničím neliší. Mohli bychom tedy náš dotaz přepsat do této podoby (a vrátil by totéž):

```
select knihy.id, knihy.nazev, druhy.nazev as
druh,
max_doba_vypujcky from druhy right join knihy
on knihy.druh = druhy.id;
```

De facto bychom si vystačili buď jen s levým, nebo jen s pravým spojením. Ale nezapomeňme, že pomocí klauzule JOIN lze spojit více než dvě tabulky. Pak může být možnost použít "levé" nebo "pravé" spojení k nezaplacení, neb můžeme tabulky vyjmenovávat v pořadí, v jakém se nám to hodí.

6.5 Vícenásobná spojení

Jak již jsme uvedli, lze spojit více než dvě tabulky. Přitom není nutné, aby byly všechny tabulky spojeny pomocí vnitřního (nebo vnějšího spojení). Je například možné spojit dvě tabulky pomocí vnitřního spojení a třetí připojit pomocí spojení vnějšího. Mohlo by to vypadat nějak takto:

```
select hlavicka_faktury.*, polozky_faktury.*,
zakaznik.nazev
from hlavicky_faktury inner join
polozky_faktury on hlavicka_fatkury.id =
polozky_faktury.hlavicka
left join zakaznik on faktura.zakaznik =
zakaznik.id;
```

V minulosti toho bylo hodně napsáno o tom, v jakém pořadí se mají jednotlivé spojované tabulky v příkazu uvádět. V dobách databázového pravěku totiž vykonával stroj spojení tupě zleva doprava. V poslední době je to jedno - DBMS mají takzvané optimizátory, které v naprosté většině případů dokáží spolehlivě odhadnout, v jakém pořadí tabulky spojit, aby byl výsledek poskytnut volající aplikaci co možná nejrychleji.

6.6 Význam spojení

Uvědomme si, že relační databáze jsou na spojování šikovně navržených tabulek postaveny. Je tedy nesmírně důležité dobře si osvojit syntaxi spojení a pokud možno se naučit psát příslušný kód z hlavy. Vizualní nástroje, které umožňují modelovat spojení pomocí klikání

myší bez znalostí odpovídajícího kódu jsou sice poměrně produktivní, ale často si takový databázový programátor zvykne a zpohodlní. Má-li potom odladit nějaký dotaz BEZ grafických nástrojů, je to problém.

O dalších věcech souvisejících se spojením budeme mluvit v příštím díle našeho seriálu.

6.7 CROSS JOIN - kartézský součin

CROSS JOIN spojí tabulky tak, že výsledkem je jejich kartézský součin. To znamená, že pro každý řádek z jedné tabulky je ve výsledné sadě záznamů vytvořena kombinace se všemi řádky z tabulky druhé. Má-li tedy první tabulka 5 řádků a druhá 2 řadky, bude ve výsledné množině řádků deset.

6.8 Příklad

Máme dvě tabulky

```
CREATE TABLE `druzstvo1` (  
  `cislo` int(11) NOT NULL default '0',  
  `jmeno` varchar(50) collate latin2_czech_cs NOT NULL default  
  ''
```

```
) ENGINE=MyISAM DEFAULT CHARSET=latin2  
COLLATE=latin2_czech_cs;
```

```
INSERT INTO `druzstvo1` VALUES (1, 'Novák Jan');
```

```
INSERT INTO `druzstvo1` VALUES (2, 'Kučera Petr');
```

```
CREATE TABLE `druzstvo2` (  
  `cislo` int(11) NOT NULL default '0',  
  `jmeno` varchar(50) collate latin2_czech_cs NOT NULL default  
  ''
```

```
) ENGINE=MyISAM DEFAULT CHARSET=latin2  
COLLATE=latin2_czech_cs;
```

```
INSERT INTO `druzstvo2` VALUES (1, 'Zuda Martin');
```

```
INSERT INTO `druzstvo2` VALUES (2, 'Bílek Jan');
```

```
INSERT INTO `druzstvo2` VALUES (3, 'Bartoš karel');
```

Proveďte křížový dotaz

```
select * from druzstvo1 cross join druzstvo2;
```

SQL umožňuje spojení pomocí JOIN pro teoreticky libovolné množství tabulek. Syntaxe je následující:

```
SELECT sloupce FROM (((tabulka1 [INNER|LEFT|OUTER|...] JOIN tabulka2 ON sloupec1=sloupec2)
[INNER|LEFT|OUTER|...] JOIN tabulka3 ON sloupec1nebo2 = sloupec3)
[INNER|LEFT|OUTER|...] JOIN tabulka4 ON sloupec1nebo2nebo3 = sloupec4)
[INNER|LEFT|OUTER|...] JOIN tabulka5 ON sloupec1nebo2nebo3nebo4 = sloupec5)
```

Příklad

```
SELECT table1 . columnName , table3 . columnName
FROM table1
    inner join table2
        ON table1 . primarykey = table2 . foreignkey
    inner join table3
        ON table2 . primarykey = table3 . foreignkey
```

```
select Company . Company_Id , Company . Company_Name ,
    Invoice_Details . Invoice_No , Product_Details . Price
from Company inner join Invoice_Details
    on Company . Company_Id = Invoice_Details . Company_Id
    inner join Product_Details
        on Invoice_Details . Invoice_No = Product_Details .
Invoice_No
where Price = '70000' ;
```

7 Formát PDF

Základní informace naleznete na adrese: <http://www.fpdf.org/>.

- Portable Document Format
- Vznik v roce 1993, firma Adobe
- Částečně vychází z PostScriptu
- Od roku 2008 otevřený standard
- Norma ISO 32000-1:2008
- Podpora textu, rastrových i vektorových obrazů, 3D objekty, JavaScript, plug-iny
- Čtečky Adobe Reader, Foxit, ...
- Proč PDF v PHP?

Pro PHP 8,2 je nutné stáhnout poslední verzi PDF (**v1.86** (2023-06-25) je k dispozici v souborech ke stažení).

Generování nový font (aby měl češtinu)

<http://www.fpdf.org/makefont>

<https://www.youtube.com/watch?v=v63RjRqIzVE>

Nezapomenout změnit kódování na cp1250

Příklad

```
<?php
```

```
require('fpdf.php');
```

```
require("connect.php");
```

```
mysqli_set_charset($spojeni, "cp1250");
```

```
$sql = "select sklad.cislo as scislo, (sklad.nazev) as nazevSkladu , (sklad.cena) as scena ,  
(mistnost.popis) as popisMistnosti FROM sklad, mistnost where  
sklad.mistnost=mistnost.id_mistnosti";
```

```
$vysledek = mysqli_query($spojeni, $sql);
```

```
$zaznam = mysqli_fetch_assoc($vysledek);
```

```
$pdf = new FPDF();
```

```

//$pdf->AddPage();
//$pdf->Addfont('calibri','', 'calibri.php')

//$pdf->SetFont('calibri','',16);
$pdf->AddFont('Calibri','', 'calibri.php');
$pdf->AddPage();
$pdf->SetFont('Calibri','',16);

$pdf->SetFont('Arial','B',16);
$pdf->Cell(20,10,"Číslo",1);
$pdf->Cell(60,10,"Název",1);
$pdf->Cell(40,10,"Cena",1);
$pdf->Cell(40,10,"Místnost",1);

$pdf->Ln();

while ($zaznam = mysqli_fetch_assoc($vysledek)):
    $pdf->Cell(20,10,$zaznam["scislo"],1);
    $pdf->Cell(60,10,$zaznam["nazevSkladu"],1);
    $pdf->Cell(40,10,$zaznam["scena"],1);
    $pdf->Cell(40,10,$zaznam["popisMistnosti"],1);

    $pdf->Ln();

endwhile;

mysqli_close($spojeni);
$pdf->Output();
?>

```

7.1

7.2 FPDF –Základní funkce

- Přidání stránky
- AddPage([string orientation [, mixed format]])
- Přidání a výběr písma
- AddFont(string family [, string style [, string file]])
- SetFont(string family [, string style [, float size]])
- Vyrenderování dokumentu
- string Output([string name, string dest])
Dest: I –otevřít v prohlížeči, D –nabídnout ke stáhnutí, F –uložit na disk, S –vrátí v textové podobě

7.3 FPDF –Buňky

Jednořádková buňka

- Cell(float w [, float h [, string txt [, mixed border [, int ln [, string align [, boolean fill [, mixed link]]]]]]))

Víceřádková buňka

- MultiCell(float w, float h, string txt [, mixed border [, string align [, boolean fill]])

Barvy

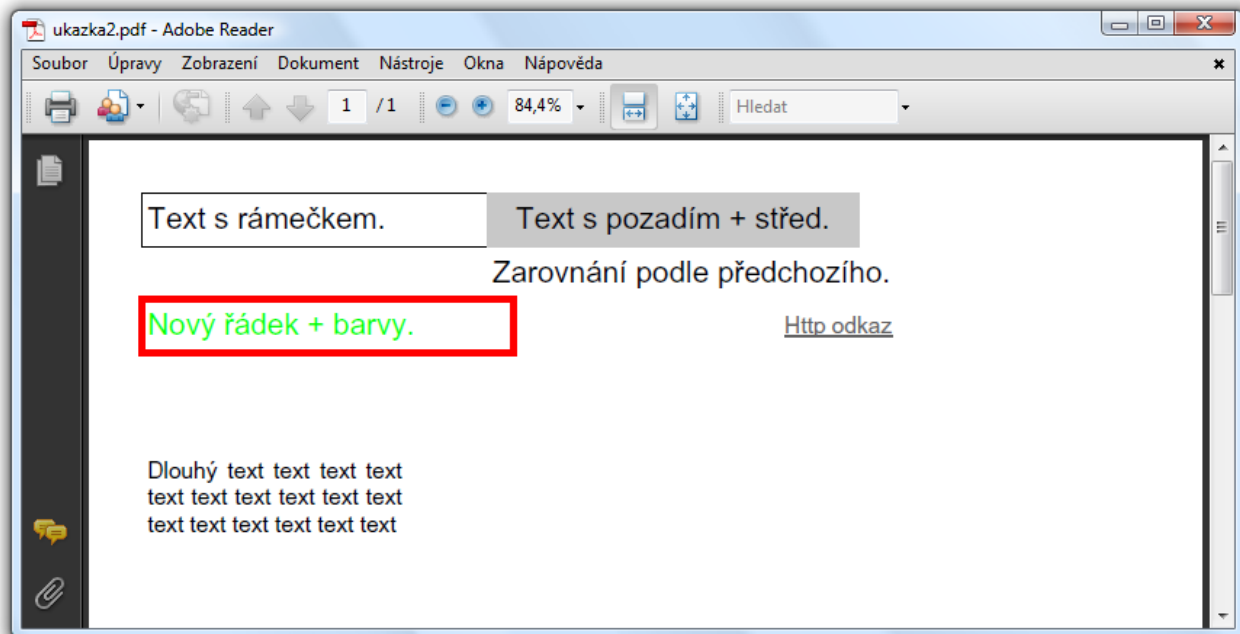
- SetTextColor(int r [, int g, int b])
- SetFillColor(), SetDrawColor()

Prázdný řádek

- `Ln([float h])`

7.4 FPDF –Buňky –Kód 1/2

[illegible]



7.5 FPDF –Záhlaví/zápatí

- Vytvoříme novou třídu, která dědí z FPDF
- Přepíšeme metody Header() a Footer()

```
<?php require('fpdf.php');  
classPDFextendsFPDF{ functionHeader(){ $this->Image('fei-new.jpg',10,10,15);  
$this->AddFont('Arial');  
$this->SetFont('Arial');  
$this->Cell(0,8,'Vysoká škola polytechnická',0,2,'R');  
$this->Cell(0,8,'FEI',0,0,'R');  
$this->Line(10,28,200,28); }  
functionFooter(){ $this->SetY(-15); $this->SetFont('Arial','',8);  
$this->Cell(0,10,'Stránka'.$this->PageNo().'{s}',0,0,'L');  
$this->Cell(0,10,'Datum'.date("j.n.YH:i:s"),0,0,'R'); }} //Použití $pdf=newPDF();  
$pdf->AliasNbPages('{s}');  
$pdf->AddPage();  
$pdf->AddPage();  
$pdf->AddPage();  
$pdf->Output(); ?>
```

Příkazy PDF

[AddFont](#) - přidání nového fontu
[AddLink](#) - vytvoření vnitřního odkazu
[AddPage](#) - přidání nové stránky
[AliasNbPages](#) - definujte alias pro počet stránek
[Cell](#) - tisk textové buňky : PŘELOŽIT
[Close](#) - uzavře dokument
[Error](#) - kritická chyba : PŘELOŽIT
[Footer](#) - patička stránky : PŘELOŽIT
[FPDF](#) - konstruktor : PŘELOŽIT
[GetStringWidth](#) - spočítá délku řetězce
[GetX](#) - získá aktuální pozici X
[GetY](#) - získá aktuální pozici Y
[Header](#) - hlavička stránky : PŘELOŽIT
[Image](#) - vloží obrázek : PŘELOŽIT
[Line](#) - nakreslí linku
[Link](#) - vloží odkaz : PŘELOŽIT
[Ln](#) - zalomení řádku : PŘELOŽIT
[MultiCell](#) - tisk textové buňky se zalomením řádků : PŘELOŽIT
[Output](#) - uloží nebo odešle dokument : PŘELOŽIT
[PageNo](#) - číslo stránky
[Rect](#) - nakreslí obdelník
[SetAuthor](#) - nastaví autora dokumentu
[SetAutoPageBreak](#) - nastaví automatický mód zalomení stánky
[SetCompression](#) - zapne nebo vypne kompresi
[SetCreator](#) - nastaví autora dokumentu
[SetDisplayMode](#) - nastaví režim zobrazení : PŘELOŽIT
[SetDrawColor](#) - nastaví barvu linky
[SetFillColor](#) - nastaví barvu výplně
[SetFont](#) - nastaví font
[SetFontSize](#) - nastaví velikost fontu
[SetKeywords](#) - sloučí klíčové slova s dokumentem
[SetLeftMargin](#) - nastaví levý okraj : PŘELOŽIT
[SetLineWidth](#) - nastaví šířku linky
[SetLink](#) - nastaví cíl interního odkazu
[SetMargins](#) - nastaví okraje
[SetRightMargin](#) - nastaví pravý okraj
[SetSubject](#) - nastaví subjekt dokumentu
[SetTextColor](#) - nastaví barvu textu
[SetTitle](#) - nastaví název dokumentu
[SetTopMargin](#) - nastaví horní okraj
[SetX](#) - nastaví aktuální pozici X
[SetXY](#) - nastaví aktuální pozici X a Y
[SetY](#) - nastaví aktuální pozici Y
[Text](#) - tisk textu : PŘELOŽIT
[Write](#) - tisk "plovoucího" textu :

<?php

```

require("../tfpdf/tfpdf.php");

$pdf = new tFPDF();
$pdf=new tFPDF("P","pt","A4");
//$pdf=new FPDF();
$pdf->AddPage();

$pdf->AddFont('arial','', 'arial.php');
//$pdf->SetFont('Arial','B',16);
$pdf->SetFont('arial','',18);

$text="ŘŤĎÁČěšřčěčáržčřžýčí";

$pdf->Cell(40,10,$text);
$pdf->Output();
?>

```

7.6 Praktické příklady více tabulek a PDF

Datový model



```
<?php
```

```

require("connect.php");
@mysql_query("SET CHARACTER SET utf8");

@mysql_query("SET NAMES utf8");

```

```
$vysledek = mysql_query("select sklad2.cislo, (sklad2.nazev) as nazevSkladu, sklad2.cena,  
(mistnosti2.nazev) as nazevMistnosti FROM sklad2, mistnosti2 where  
sklad2.mistnost=mistnosti2.id_mistnost");  
//$vysledek = mysql_query("select * FROM sklad2,mistnosti2 where sklad2.mistnost =  
mistnosti2.id_mistnost");
```

```
echo "<B>vypis hodnot ze 2 tabulek</B><BR>";  
while ($zaznam = mysql_fetch_Array($vysledek)):  
    echo "<BR> cislo: ".$zaznam["cislo"]."<BR> ";  
    echo "nazev:      ".$zaznam["nazevSkladu"]."<BR> ";  
    echo "    cena: ".$zaznam["cena"]."<BR>";  
    echo "mistnist: ".$zaznam["nazevMistnosti"]." <BR><BR>";  
  
endwhile;
```

?>

vypis hodnot ze 2 tabulek

cislo: 5
nazev: dfgdf
cena: 121
mistnist: 1.A

cislo: 3
nazev: ghjkghjk
cena: 251
mistnist: 1.B

cislo: 2
nazev: fgh
cena: 21321
mistnist: 3.A

cislo: 1
nazev: čžřčýžřýž
cena: 12
mistnist: 4.A

Výpis v pdf

```
<?php
```

```
<?php
```

```
require("connect.php");
```

```
require("tfpdf/tfpdf.php");
```

```
$sql = "select * from sklad";
```

```
$vysledek = mysqli_query($spojeni, $sql);
```

```
//      $nalezene = mysqli_query("select * from sklad2 ");
```

```
$pdf = new tFPDF();
```

```
$pdf->AddFont('DejaVu','', 'DejaVuSansCondensed.ttf', true);
```

```
$pdf->SetFont('DejaVu','', 10);
```

```
$pdf->AddPage();
```

```
$pdf->Cell(20,10,"Číslo",1);
```

```
$pdf->Cell(20,10,"Název",1);
```

```
$pdf->Cell(20,10,"Cena",1);
```

```
$pdf->Cell(20,10,"Místoprost",1);
```

```
$pdf->Ln();
```

```
while($radek=mysqli_fetch_row($vysledek))
```

```
{
```

```
    //pdf->Cell(20,10,'aaaa',1);
```

```
    $pdf->Cell(20,10,$radek[0],0);
```

```
    $pdf->Cell(20,10,$radek[1],0);
```

```
$pdf->Cell(20,10,$radek[2],0);
```

```
$pdf->Cell(20,10,$radek[3],0);
```

```
        $pdf->Ln();  
    }
```

```
$pdf->Output();
```

```
mysqli_close($connect);
```

1	????	12	4
2	fgh	21321	3
3	ghjkghjk	251	2
5	dfgdf	121	1

?>

8 Zabezpečení stránek

8.1 Zabezpečení pomocí hlavičky

```
<?php
if (isset($_SERVER['PHP_AUTH_USER']) && isset($_SERVER['PHP_AUTH_PW']) &&
$_SERVER['PHP_AUTH_USER'] == 'uzivatel' && $_SERVER['PHP_AUTH_PW'] == 'heslo') { // vse v
poradku
    echo 'Prihlaseni probehlo uspesne.';
}
else { // chyba prihlaseni
    header('HTTP/1.0 401 Unauthorized');
    header('WWW-Authenticate: Basic realm="Login"');
    echo 'Chyba prihlaseni - zadejte platne uzivatelske jmeno a heslo!';
    exit;
}
?>
```

8.2 Session

Session řeší problém bezstavovosti protokolu HTTP, udržíme pomocí ní informace o stavu aplikace a práci uživatele sní.

Jsou to proměnné, ukládání přímo na serveru.

- v session můžeme mít také spoustu proměnných (parametrů), které bychom jinak museli předávat (a tedy i přenášet) v URL, což je nejen pracné, ale v případě některých parametrů také ne zrovna bezpečné.
- pokud máme proměnné v relaci, tyto se vůbec nepřenášejí mezi serverem a počítačem klienta, skript na serveru je dostává přímo z interpretu PHP.
- doporučeno je využití superglobální proměnné `$_SESSION`.

8.3 Základní příkazy

funkce	význam
session_start()	Inicializuje (nastartuje) relaci.
session_destroy()	Odstraní všechna data relace (např. při odhlašování uživatele).
\$_SESSION['promenna']	Zápis superglobální proměnné pro přístup k proměnné <i>promenna</i> v relaci, dřívější funkce <i>session_register</i> je nedoporučená.
unset(\$_SESSION['promenna'])	Zrušení hodnoty proměnné <i>promenna</i> v session, dřívější funkce <i>session_unregister</i> je nedoporučená.
SID	Konstanta – v případě, že je relace zahájena a nepracují cookies, obsahuje identifikátor relace pro předání do URL; pokud pracují cookies, konstanta neexistuje, jméno této konstanty může být změněno nastavením v <i>php.ini</i> .

8.4

8.5 Praktický příklad session

header("Location: private/index.php"); - redirect to index.php

All application consist of 3 files

1. Files – shows input form (inserting login and passsword)
2. Files - runs protect pages – only for logged user
3. Files user logs off and session will be end

User authorization:

Login.php

```
<?
```

```
$login="Yoda"; //uživatelské jméno
```

```
$pass="jedi"; //heslo
```

```
//Pokud byl odeslán formulář (v proměnné action je hodnota validace)
```

```
if ($_GET['action']=='validate'){
```

```
    //a pokud odpovídají přihlašovací údaje
```

```
    if(($_POST['user']==$login)&&($_POST['passwd']==$pass)){
```

```
        session_start();
```

```
        header("Cache-control: private"); // požadováno u některých prohlížečů
```

```
        //zaregistruje proměnou user_is_logged a nastaví ji na 1
```

```
        $_SESSION["user_is_logged"] = 1;
```



```

        //a poslena úvodní soubor chráněné sekce
        header("Location: private/index.php");
        exit;
    }
}
?>

//inserting form

<form action="./login.php?action=validate" method="post">
    <table>
        <tr><td>Uživatel </td><td><input type="text" name="user" /></td></tr>
        <tr><td>Heslo </td><td><input type="password" name="passwd"
/></td></tr>
        <tr><td colspan="2"><input type="submit" value="...: Okej :..." />
        <input type="reset" value="...: Storno :..." /></td></tr>
    </table>
</form>

protection.php Check

```

Protect pages Show

```

//specify file, which will be included when user is not logged-in

protection.php
<?
    //nezapomeň nastartovat session!
    session_start();
    header("Cache-control: private");
    //Pokud není uživatel přihlášen, pošleme mu přihlašovací formulář
    if ($_SESSION["user_is_logged"] != 1){
        header("Location: login.php");
        exit();
    }
?>

```

User logout

logout.php

```
<?
    session_start();
// zruší všechny proměnné
    $_SESSION = array();//Clean all registry
    session_destroy();
if ($_SESSION["user_is_logged"]){
    echo "FATAL ERROR: Cannot terminate session!";
} else {
    header("Location: login.php");
}
?>
```

Finish

Insert in every protect file

```
require "protection.php";
```

8.6 Šifrování hesel pomocí md5

```
<?php
$heslo="tajne";
$zasifrovane=md5($heslo);
Echo "heslo “. $heslo. “je zasifrovane takto .” $zasifrovane”;
?>
```

9 Čeština a database

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
```

```
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />

<title>CR_dotaznik</title>
</head>
<body>

<?
```

```
@$mysql = MySQL_Connect("localhost", "smrcka", "heslo");
@MySQL_Select_DB("smrcka");
@mysql_query("SET CHARACTER SET utf8");
```

```
    @mysql_query("SET NAMES utf8");
```

```
$vysledek = mysql_query("select * from form ");
$pocet=mysql_num_rows($vysledek);
echo "pocet polozek je: " . $pocet."<br><br>";
```

9.1 Poznámka

Addslashes

Quote string with slashes

Returns the escaped string

```
<?php
$str = "Is your name O'reilly?";
```

```
// Outputs: Is your name O\'reilly?
echo addslashes($str);
?>
```

10

11 Urlencode

Returns a string in which all non-alphanumeric characters except `-._` have been replaced with a percent (%)